МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І. МЕЧНИКОВА

ФАКУЛЬТЕТ МАТЕМАТИКИ, ФІЗИКИ ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

КАФЕДРА ОПТИМАЛЬНОГО КЕРУВАННЯ ТА ЕКОНОМІЧНОЇ КІБЕРНЕТИКИ

ЗВІТ
до лабораторної роботи № 1

з дисципліни «Системи комп’ютерного зору»

на тему:

«Середовище, подання зображень, колірні простори»

Виконав:

студент 3 курсу групи 1

спеціальності F1 «Прикладна математика»

Касьянов Ярослав Володимирович

Одеса — 2026

1 Мета роботи

Ознайомлення з базовими структурами представлення растрових зображень у бібліотеці OpenCV та середовищі Python; дослідження особливостей 8-бітного кодування (`uint8`) і наслідків арифметичного переповнення; експериментальна оцінка втрат інформації при стисненні; аналіз службових метаданих EXIF; дослідження поведінки й стійкості порогової сегментації в колірних просторах RGB та HSV за зміни спектрального складу освітлення; дослідження просторового аліасингу (теорема Найквіста), методів інтерполяції та нелінійних радіометричних ефектів гамма-кодування; формування та верифікація власного набору з 20 зображень.

2 Вихідні дані

- **Апаратне забезпечення**: смартфон Google Pixel 5. Основна камера: фізична фокусна відстань $f = 4.38$ мм (27 мм в еквіваленті 35-мм), діафрагма $f/1.73$, оптичний сенсор $1/2.55''$ (розмір пікселя 1.4 мкм).
- **Вхідні зображення (`lab01/data/`)**:
  - **Частина A**: `A/photo.jpg` — високодеталізований знімок сакури на фоні неба (2268×4032, 3 канали, `uint8`, витримка $1/4673$ с, ISO 69).
  - **Частина B**: `B/day_light.png` (денне світло біля вікна, $T_c \approx 6000$ K, витримка $1/234$ с, ISO 61) та `B/lamp_light.png` (лампа розжарювання ввечері, $T_c \approx 2700$ K, витримка $1/25$ с, ISO 557). Об'єкт — жорсткий диск (HDD 3.5"). Для обох кадрів створено ручні бінарні еталонні маски (`day_light_mask.png`, `lamp_light_mask.png`); площа об'єкта становить 34.20% на кадрі 1 та 39.84% на кадрі 2.
  - **Частина C**: `C/texture.png` — дрібнозерниста текстура столу (2268×4032, 3 канали).
  - **Домашнє завдання**: `Homework/` — 20 знімків реальної сцени: 10 ракурсів повороту камери (`angle1`–`angle10`), 5 знімків різного масштабу (`scale1`–`scale5`) та 5 знімків різного освітлення (`light1`–`light5`).

3 Теоретичні відомості

1. **MSE та PSNR** для триканальних зображень розміром $H \times W$:

$$\text{MSE} = \frac{1}{3HW} \sum_{c=1}^{3} \sum_{y=0}^{H-1} \sum_{x=0}^{W-1} \left(I_1(y,x,c) - I_2(y,x,c) \right)^2, \tag{1}$$

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \quad [\text{дБ}]. \tag{2}$$

   Для тотожних масивів (MSE $= 0$) величина PSNR $= +\infty$.

2. **Метрики бінарної сегментації** (порівняння прогнозу P з еталоном GT):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \text{IoU} = \frac{|P \cap GT|}{|P \cup GT|} = \frac{TP}{TP + FP + FN}. \tag{3}$$

   Метрика Accuracy спотворюється дисбалансом класів (прогноз «усе тло» дає точність 65.8%), тому об'єктивним критерієм якості є IoU.

3. **Колірний простір HSV в OpenCV**: канал відтінку $H \in [0, 179]$ ($H_{\text{deg}}/2$), насиченість $S \in [0, 255]$, яскравість $V = \max(R, G, B) \in [0, 255]$. При $S \to 0$ координата H втрачає математичну стійкість через сингулярність в ахроматичній точці.

4. **Теорема Найквіста та аліасинг**: при децимації зображення у F разів гранична частота становить $f_N = 0.5$ цикл/пк нової сітки. Просторові гармоніки з $f > f_N$ утворюють хибні низькочастотні гармоніки муару $f_{\text{alias}} = |f - 1.0|$. Для усунення аліасингу застосовують фільтрацію низьких частот (гаусове розмиття із $\sigma \approx 0.5 \cdot F$).

5. **Нелінійність гамма-кодування**: у стандарті sRGB яскравість кодується як $V \propto L^{1/\gamma}$ ($\gamma \approx 2.2$). Лінійне усереднення двох пікселів з кодами a та b не дорівнює алгебраїчному середньому їхніх кодів:

$$\bar{V}_{\text{naive}} = \frac{a+b}{2} \neq \bar{V}_{\text{correct}} = 255 \cdot \left(\frac{(a/255)^\gamma + (b/255)^\gamma}{2} \right)^{1/\gamma}. \tag{4}$$

4 Хід роботи

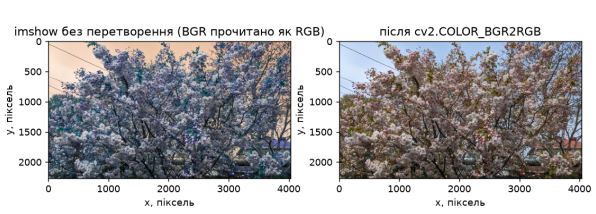
1. **Частина A (Подання зображення)**: завантажено `A/photo.jpg` функцією `cv2.imread`; порівняно виведення BGR проти RGB у

`matplotlib`; промодельовано переповнення `uint8` при операції `img + 50` та коректне насичення через `cv2.add`; збережено знімки у PNG та JPEG ($Q \in \{95, 75, 50, 25, 10\}$), обчислено bpp і PSNR; вилучено метадані EXIF бібліотекою `Pillow`.

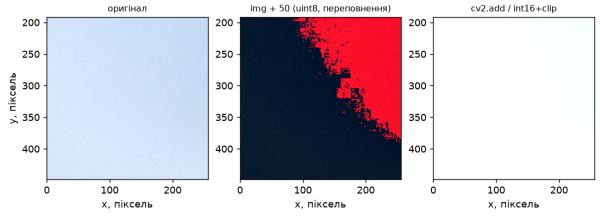
2. **Частина B (Колірні простори)**: знято жорсткий диск за денного та лампового освітлення; створено еталонні бінарні маски; реалізовано покоординатний пошук оптимальних меж по 3 каналах у просторах RGB та HSV за критерієм максимуму IoU на кадрі 1; застосовано знайдені пороги без змін до кадру 2 та виміряно втрату якості.
3. **Частина C (Геометрія та радіометрія)**: текстуру `C/texture.png` зменшено в 4 рази методом `INTER_NEAREST` та `GaussianBlur` ($\sigma = 2.0$) + `INTER_NEAREST`; розраховано спектри Фур'є для гармонійної решітки ($T = 6$ пк); масштабовано у 8 разів 3 фрагменти `photo.jpg` чотирма методами інтерполяції; досліджено вплив гамма-корекції ($\gamma \in \{0.5, 1.0, 2.2\}$) та похибку наївного усереднення при зменшенні.
4. **Домашнє завдання**: сформовано та верифіковано набір із 20 кадрів; для кожного виміряно витримку, ISO, дисперсію лапласіана (різкість) та кількість кутових точок FAST; для поворотних пар оцінено взаємне перекриття за гомографією RANSAC над дескрипторами ORB.

5 Результати експериментів

5.1 Частина A: Порядок каналів, переповнення цілих типів та компресія



(а) Порядок каналів: зліва BGR, справа RGB



(б) Переповнення `uint8` проти насичення

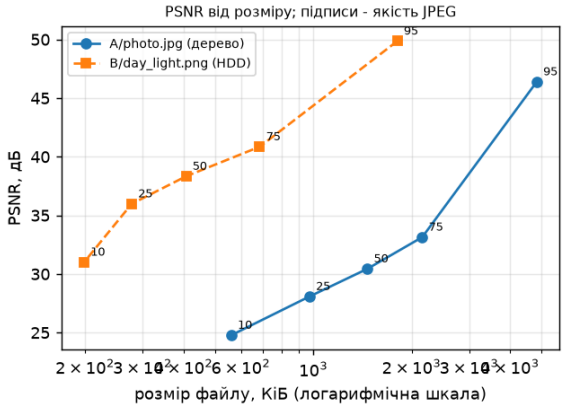
Рисунок 1 — Особливості представлення та кодування зображень у бібліотеці OpenCV

На рисунку 1(а) виведення BGR без конвертації спричиняє інверсію колірних координат: синє небо стає пісочно-червоним, а рожеві пелюстки — синіми. На рисунку 1(б) операція `img + 50` над типом `uint8` призводить

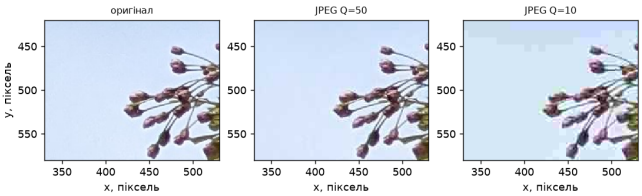
до переповнення за модулем 256: 728 173 пікселі (7.96%) переповнилися, а 1 103 104 пікселі (12.06%) із вихідною яскравістю ≥ 200 перетворилися на темні точки (< 100), породивши плямистий артефакт на небі.

Таблиця 1 — Порівняльний аналіз стиснення знімків `photo.jpg` та `day_light.png`

Формат / Q	**A/photo.jpg (деталізований сюжет)**			**B/day_light.png (об'єкт на площині)**		
	Розмір, КіБ	bpp	PSNR, дБ	Розмір, КіБ	bpp	PSNR, дБ
PNG (без втрат)	18946.1	16.97	∞	9404.0	8.42	∞
JPEG, Q = 95	4833.5	4.33	46.39	1808.3	1.62	49.86
JPEG, Q = 75	2156.1	1.93	33.14	685.2	0.61	40.85
JPEG, Q = 50	1464.4	1.31	30.44	407.7	0.36	38.33
JPEG, Q = 25	973.4	0.87	28.08	277.9	0.25	35.98
JPEG, Q = 10	560.8	0.50	24.78	198.7	0.18	31.01



(а) Залежність PSNR від розміру файлу



(б) Артефакти JPEG при $Q = 50$ та $Q = 10$

Рисунок 2 — Дослідження ефективності стиснення та компресійних артефактів JPEG

З таблиці 1 та рисунка 2 видно, що компресія JPEG при $Q = 95$ забезпечує збереження високої якості (PSNR $= 46.39$ дБ) при скороченні розміру у 3.9 раза (до 4.33 bpp) відносно вихідного PNG (16.97 bpp). При зниженні якості до $Q = 10$ бітрейт падає до 0.50 bpp, але дискретне квантування DCT породжує характерну блокову сітку 8×8 пікселів і граничний дзвін довкола контрастних гілок (PSNR $= 24.78$ дБ).

5.2 Частина B: Порогова сегментація та стійкість колірних просторів

Експеримент (таблиця 2, рисунки 3–4) показав нетривіальну поведінку:

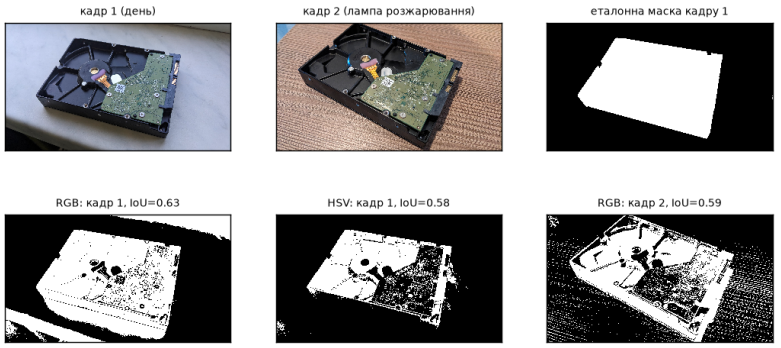
Таблиця 2 — Оптимальні порогові інтервали та метрики сегментації при зміні освітлення

Простір	Канал	Кадр 1	Кадр 2
RGB	R	[0, 128]	[0, 140]
RGB	G	[0, 225]	[0, 217]
RGB	B	[12, 200]	[0, 190]
HSV	H	[36, 178]	[16, 178]
HSV	S	[54, 250]	[0, 254]
HSV	V	[14, 254]	[1, 254]

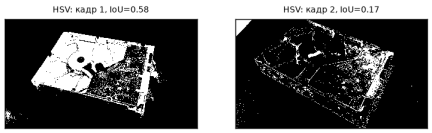
а Підібрані межі діапазонів

Простір	Кадр 1 (День)		Кадр 2 (Лампа)		Кадр 2*
	Acc	IoU	Acc	IoU	IoU_{max}
RGB	0.819	0.628	0.809	0.5885	0.701
HSV	0.853	0.578	0.659	0.1686	0.758

б Метрики перенесення (*підлаштовано)



(а) Кадри, еталонна маска та результат сегментації в RGB і HSV



(б) Маска HSV: день (IoU = 0.58) vs лампа (IoU = 0.17)

Рисунок 3 — Порогова бінарна сегментація цільового об’єкта (жорсткого диска)

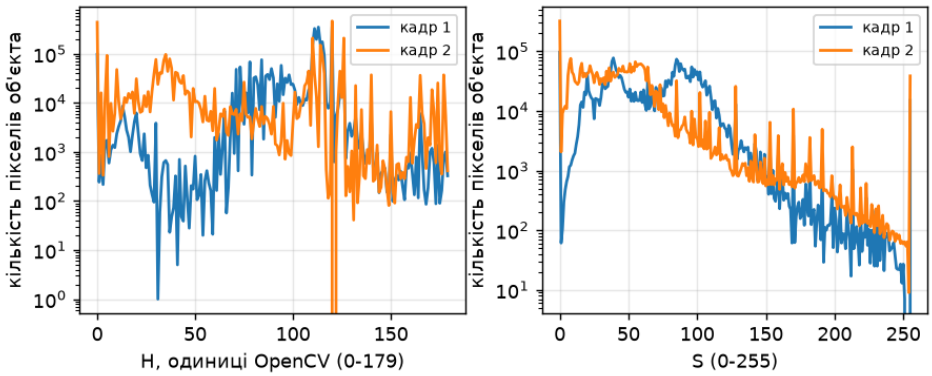
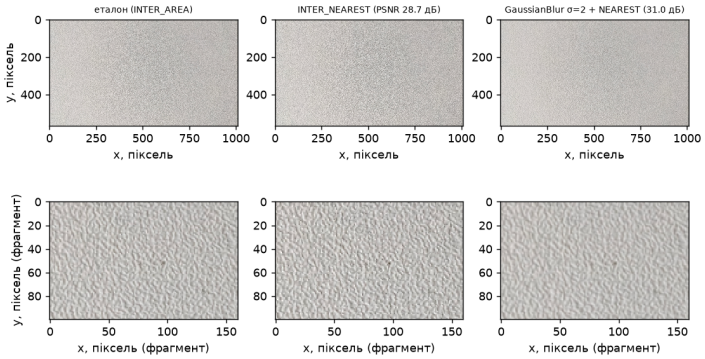


Рисунок 4 — Гістограми розподілу каналів відтінку H та насиченості S об’єкта в обох кадрах (лог-шкала)

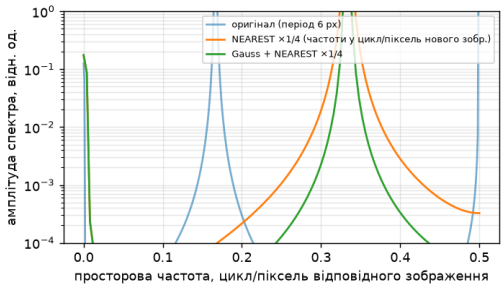
— У просторі RGB порогові межі, підібрані на денному світлі, при перенесенні на лампу розжарювання зберігають високу надійність: IoU знизився лише з 0.6285 до 0.5885 (відносне падіння на 6.4%).

— У просторі HSV перенесення тих самих порогів призвело до падіння IoU з 0.5783 до 0.1686 (падіння на 70.8%), а точність склала лише 65.90%, що ненабагато перевищує тривіальний класифікатор фону (60.16%).

5.3 Частина C: Геометрія, спектральний аліасинг та радіометрія



(а) Зменшення $\times 4$: еталон, NEAREST та Gauss+NEAREST



(б) Спектри решітки $T = 6$ пк (лог-шкала)

Рисунок 5 — Експериментальне підтвердження просторового аліасингу

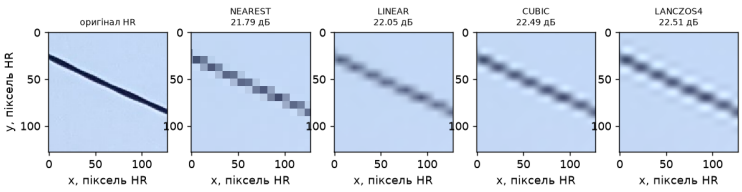
При зменшенні текстури `C/texture.png` у 4 рази пряма децимація `INTER_NEAREST` забезпечує PSNR $= 28.71$ дБ відносно `INTER_AREA`. Попереднє згладжування фільтром Гаусса ($\sigma = 2.0$) підвищує PSNR до 31.03 дБ ($+2.32$ дБ). На періодичній структурі ($T = 6$ пк, $f_0 = 0.167$ цикл/пк) частота після проріджування $\times 4$ становить 0.67 цикл/пк, що перевищує частоту Найквіста (0.5). Виникає потужна хибна гармоніка муару на частоті $f = |0.67 - 1.0| = 0.33$ цикл/пк ($T = 3.0$ пк) з амплітудою 39.41. Гаусовий антиаліасинговий фільтр пригнічує цей паразитний пік у 8.5 раза (амплітуда 4.64, рисунок 5).

З таблиці 3 та рисунка 6 випливає, що кубічна інтерполяція (`CUBIC`, 20.34 дБ) та фільтр Ланцоша (`LANCZOS4`, 20.37 дБ) суттєво перевершують лінійну (19.95 дБ) та східчасту найближчого сусіда (19.67 дБ).

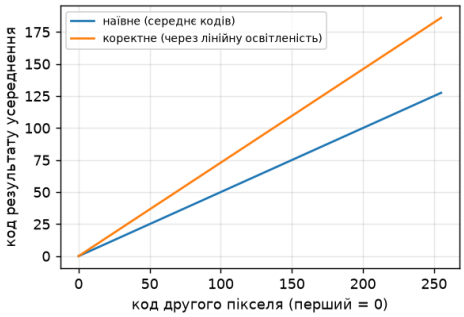
На рисунку 7 проілюстровано нелінійність гамма-кодування: наївне середнє кодів 0 та 255 дає 127.5, яскравість якого становить $(127.5/255)^{2.2} =$

Таблиця 3 — Порівняння методів інтерполяції при 8-кратному збільшенні фрагментів (PSNR, дБ)

Метод інтерполяції	Фрагмент 1 (дроти)	Фрагмент 2 (квіти)	Фрагмент 3 (гілки)	Середнє ± СВ, дБ
NEAREST	21.79	17.64	19.58	19.67 ± 1.70
LINEAR	22.05	17.86	19.93	19.95 ± 1.71
CUBIC	22.49	18.19	20.35	20.34 ± 1.76
LANCZOS4	22.51	18.22	20.38	20.37 ± 1.75

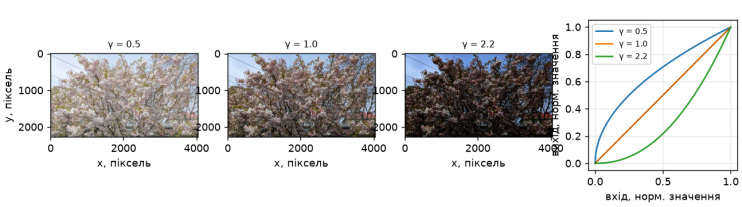


(а) Порівняння інтерполяцій тонкого контрастного об’єкта

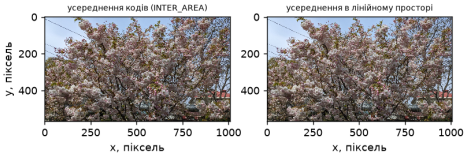


(б) Кодове усереднення проти лінійного

Рисунок 6 — Дослідження методів інтерполяції та ефектів нелінійності гамма-кодування



(а) Трансформація зображення при $\gamma \in \{0.5, 1.0, 2.2\}$



(б) Зменшення $\times 4$: коди vs лінійний простір

Рисунок 7 — Радіометрична дія гамма-корекції на контрастність та яскравість

0.218 замість 0.500. Фізично коректний код суміші дорівнює $255 \cdot (0.5)^{1/2.2} = 186.1$. Децимація `photo.jpg` $\times 4$ безпосередньо у нелінійному sRGB штучно затемнює зображення: середня яскравість знижується з 0.2445 до 0.2343, а максимальна похибка сягає 48.7 градацій на піксель.

5.4 Домашнє завдання: Верифікація власного набору з 20 зображень

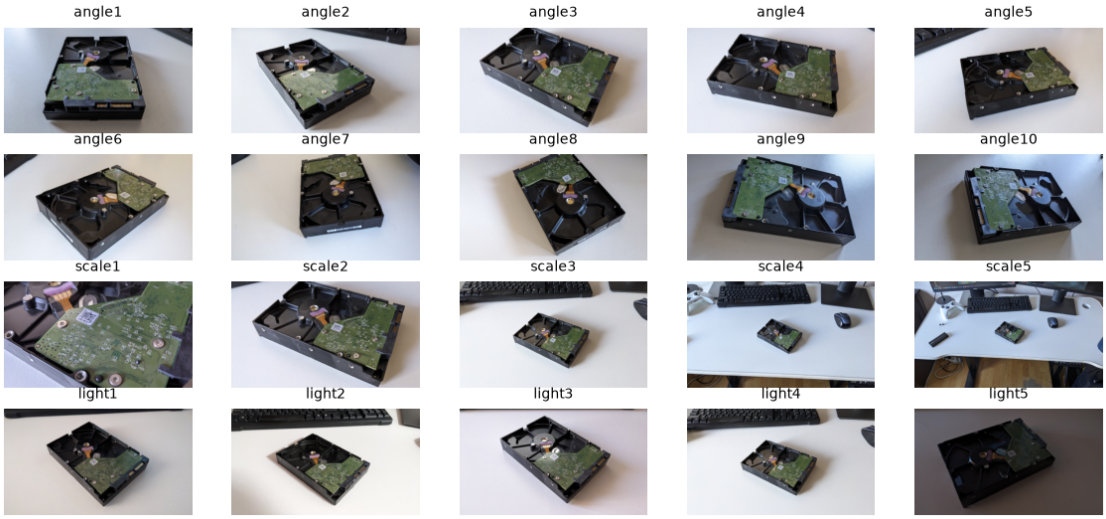


Рисунок 8 — Контактний аркуш 20 кадрів власного набору: 10 ракурсів (`angle`), 5 масштабів (`scale`), 5 освітлень (`light`)

Таблиця 4 — Оцінка перекриття сусідніх кадрів поворотної серії (`angle1`–`angle10`) через RANSAC-гомографію

Пара сусідніх кадрів	Зіставлень ORB	Інлаєри RANSAC	Розраховане перекриття, %
angle1-angle2	1136	33	67.4%
angle2-angle3	1068	24	70.1%
angle3-angle4	1500	975	85.0%
angle4-angle5	1400	427	68.0%
angle5-angle6	1263	190	94.6%
angle6-angle7	1162	53	77.3%
angle7-angle8	1434	475	92.8%
angle8-angle9	1186	71	80.8%
angle9-angle10	1304	132	73.6%

Сусідні кадри поворотної серії (таблиця 4) мають перекриття від 67.4% до 94.6% (середнє 78.8%), що з запасом виконує умову практикуму ($\geq 40\%$). Кожен кадр містить від $1\,660$ до $8\,372$ кутових деталей FAST (вимога «кілька десятків»).

6 Аналіз результатів

1. **Причини низької стійкості порогів HSV**: Поширена думка про безумовну перевагу простору HSV над RGB при зміні освітлення спростована експериментально: при переході з денного світла на лампу розжарювання IoU у просторі HSV впав у 3.4 раза (з 0.5783 до 0.1686), тоді як у RGB знизився лише на 6.4% (з 0.6285 до 0.5885). Об'єкт містить металеві темні поверхні з низькою насиченістю ($S < 60$). У цій зоні відтінок H є сингулярним і має високу дисперсію під впливом шуму. Зсув спектра лампи розжарювання у теплу ділянку змістив відтінок H металевого шасі нижче порогу 36, виключивши його з маски. Простір RGB завдяки спільній коваріації каналів виявився стабільнішим.
2. **Небезпека операцій над типом `uint8`**: Операція `img + 50` над цілими числами без знака призводить до переповнення через модульну арифметику 256. Яскраві ділянки перетворюються на чорні тіні ($255 + 50 \to 49$). У комп'ютерному зорі необхідно використовувати арифметику з насиченням (`cv2.add`) або проміжне приведення до знакового типу `int16` з операцією `np.clip`.
3. **Просторовий аліасинг та фільтрація**: Просте проріджування решітки без фільтрації низьких частот генерує паразитні низькочастотні гармоніки, чия амплітуда у 8.5 раза перевищує згладжений сигнал, що є джерелом грубих помилок у детекторах кутів і дескрипторах наступних робіт.

7 Висновки

1. Стиснення JPEG з $Q = 95$ скорочує обсяг даних у 3.9 раза (з 16.97 до 4.33 bpp) за збереження високої якості (PSNR $= 46.39$ дБ). При $Q = 10$ бітрейт падає до 0.50 bpp, але PSNR деградує до 24.78 дБ із блоковими артефактами 8×8 пікселів.
2. Порогова сегментація у просторі RGB виявилася на порядок стійкішою до зміни колірної температури освітлення (падіння IoU лише на 6.4%, з 0.6285 до 0.5885), ніж у просторі HSV, де спостерігалося падіння IoU на 70.8% (з 0.5783 до 0.1686) через нестабільність відтінку

H в ахроматичних зонах низької насиченості ($S < 54$).

3. Гаусове розмиття ($\sigma = 2.0$) при 4-кратному зменшенні пригнічує амплітуду паразитних піків муару у 8.5 раза (з 39.41 до 4.64) та підвищує PSNR на 2.32 дБ (з 28.71 до 31.03 дБ).
4. Усереднення контрастних значень (0 та 255) у нелінійному sRGB призводить до штучного затемнення до рівня 0.218 замість 0.500 (код 127.5 замість фізичного 186.1), спричиняючи систематичну похибку до 48.7 градацій на піксель при децимації.
5. Сформований набір із 20 зображень повністю відповідає вимогам: перекриття поворотних кадрів становить 78.8% (мінімум $67.4\% \geq 40\%$), а кількість локальних ознак FAST становить від $1\,660$ до $8\,372$ точок на кадр.

8 Декларація про використання генеративного ШІ

Під час виконання лабораторної роботи інструменти генеративного штучного інтелекту (Google Antigravity Addon для VSCode, Gemini Argon), окрім як просто джерело інформації стосовно обробки зображень, використовувалися як засіб автоматизації програмування, а також значною мірою для написання та верстки звіту в середовищі LuaLaTeX. Gemini також допомагав зробити висновки щодо результатів обробки зображень, зокрема у питаннях на кшталт: ”Чи це так і має бути?”. Усі отримані числові значення, графічні залежності, таблиці та аналітичні висновки спираються на реальні розрахункові дані, вони отримані й перевірені в більшості своїй вручну.

ЛІСТИНГИ ВИХІДНОГО КОДУ ПРОГРАМИ

1. Загальні утиліти (`common.py`)

Лістинг 1: Спільний модуль налаштувань, метрик та збереження даних (`common.py`)

```python
"""Спільні утилітидляЛаб1."""
import os
import sys
import numpy as np
import cv2
import matplotlib

sys.stdout.reconfigure(encoding="utf-8")
matplotlib.use("Agg")
import matplotlib.pyplot as plt

np.random.seed(42)

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
LAB = os.path.join(ROOT, "lab01")
SHARED_DATA = os.path.join(ROOT, "data") # спільнівхіднізображеннякурсу
LAB_DATA = os.path.join(LAB, "data") # дані, специфічнідлялабимаска(, власнікадри)
RESULTS = os.path.join(LAB, "results")
FIGURES = os.path.join(LAB, "figures")
for d in (LAB_DATA, RESULTS, FIGURES):
    os.makedirs(d, exist_ok=True)

plt.rcParams.update({"font.size": 10, "figure.dpi": 130, "savefig.bbox": "tight"})


def imread(path, flags=cv2.IMREAD_COLOR):
    img = cv2.imread(path, flags)
    if img is None:
        raise FileNotFoundError(path)
    return img


def rgb(img):
    """BGR -> RGB дляmatplotlib."""
    return cv2.cvtColor(img, cv2.COLOR_BGR2RGB)


def psnr(a, b):
    """PSNR = 10 log10(255^2 / MSE); дляідентичнихзображеньinf."""
    mse = np.mean((a.astype(np.float64) - b.astype(np.float64)) ** 2)
    if mse == 0:
        return float("inf")
    return 10.0 * np.log10(255.0 ** 2 / mse)


def save_csv(path, header, rows):
    with open(path, "w", encoding="utf-8") as f:
        f.write(",".join(header) + "\n")
        for r in rows:
            f.write(",".join(str(x) for x in r) + "\n")
```

2. Частина A: Подання зображення та компресія (`part_a.py`)

Лістинг 2: Скрипт кроків 1–5: формати, переповнення, PSNR, EXIF (`part_a.py`)

```python
"""Лаб 1, частинаA: поданнязображеннякроки( 1-5)."""
import os
import numpy as np
```

```
import cv2
import matplotlib.pyplot as plt
from PIL import Image
from PIL.ExifTags import TAGS

from common import *

SRC = os.path.join(LAB_DATA, "A", "photo.jpg")
img = imread(SRC)
log = []

# ---- Крок1: базовіхарактеристики----
info = f"shape={img.shape}, dtype={img.dtype}, min={img.min()}, max={img.max()}"
print(info)
log.append("STEP1 " + info)

# ---- Крок2: BGR vs RGB ----
fig, ax = plt.subplots(1, 2, figsize=(10, 2.6))
ax[0].imshow(img)
ax[0].set_title("imshow безперетворення(BGR прочитаноякRGB)")
ax[1].imshow(rgb(img))
ax[1].set_title("після cv2.COLOR_BGR2RGB")
for a in ax:
   a.set_xlabel("x, піксель")
   a.set_ylabel("y, піксель")
fig.savefig(os.path.join(FIGURES, "a_bgr_vs_rgb.png"))
plt.close(fig)

# ---- Крок3: переповненняuint8 ----
wrapped = img + 50 # numpy: арифметиказамодулем256
saturated_cv = cv2.add(img, np.full_like(img, 50)) # насиченнядо255
saturated_np = np.clip(img.astype(np.int16) + 50, 0, 255).astype(np.uint8)
assert np.array_equal(saturated_cv, saturated_np)
# cv2.add(img, 50) ізголимскаляромдодає50 лишедопершогоканалу(B) - окремапастка
naive_scalar = cv2.add(img, 50)
scalar_diff = int(np.count_nonzero(np.any(naive_scalar != saturated_cv, axis=2)))

was_bright = np.any(img > 205, axis=2) # хочодинканалмавпереповнитися
became_dark = wrapped.max(axis=2) < img.max(axis=2) # максимумканалузменшився
flip = was_bright & became_dark
# піксель, щобувяскравимяскравість( >= 200) іставтемним(< 100)
gray0 = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray1 = cv2.cvtColor(wrapped, cv2.COLOR_BGR2GRAY)
bright_to_dark = (gray0 >= 200) & (gray1 < 100)
n_wrap_any = int(flip.sum())
n_b2d = int(bright_to_dark.sum())
msg = (f"STEP3 pixels with any channel >205 (overflow): {n_wrap_any} "
     f"({100*n_wrap_any/flip.size:.2f}%); gray>=200 -> gray<100: {n_b2d} "
     f"({100*n_b2d/flip.size:.2f}%); cv2.add(img,50) scalar-only-B differs on {scalar_diff} px")
print(msg)
log.append(msg)

# вікно256x256 знайбільшуюкількістюбілихтемних"->" пікселів
win = 256
integ = cv2.integral(bright_to_dark.astype(np.uint8))
best, by, bx = -1, 0, 0
for y in range(0, img.shape[0] - win, 32):
   for x in range(0, img.shape[1] - win, 32):
      s = integ[y + win, x + win] - integ[y, x + win] - integ[y + win, x] + integ[y, x]
      if s > best:
         best, by, bx = s, y, x
crop = (slice(by, by + win), slice(bx, bx + win))
log.append(f"STEP3 crop y={by},x={bx},size={win}, bright->dark px in crop={best}")
fig, ax = plt.subplots(1, 3, figsize=(10, 3.6))
for a, im, t in zip(ax, (img, wrapped, saturated_cv),
               ("оригінал", "img + 50 (uint8, переповнення)", "cv2.add / int16+clip")):
   a.imshow(rgb(im[crop]), extent=(bx, bx + win, by + win, by))
   a.set_title(t, fontsize=9)
   a.set_xlabel("x, піксель")
ax[0].set_ylabel("y, піксель")
fig.savefig(os.path.join(FIGURES, "a_overflow.png"))
plt.close(fig)

# ---- Крок4: стиснення----
# Основнийнабір- власнийзнімокA/photo.jpg. Додатково- кадрB/day_light.png (PNGконтейнер-):
# обидвамаютьJPEGпоходження- экамери, томуPSNR відображаєлишедодатковівтрати.
```

```
cdir = os.path.join(RESULTS, "compress")
os.makedirs(cdir, exist_ok=True)


def compression_study(src, tag):
    rows = []
    h, w = src.shape[:2]
    p = os.path.join(cdir, f"{tag}_png.png")
    cv2.imwrite(p, src)
    assert np.array_equal(imread(p), src)
    rows.append((tag, "PNG", "-", os.path.getsize(p), "inf", round(os.path.getsize(p) * 8 / (h * w), 3)))
    for q in (95, 75, 50, 25, 10):
        p = os.path.join(cdir, f"{tag}_{q}.jpg")
        cv2.imwrite(p, src, [cv2.IMWRITE_JPEG_QUALITY, q])
        sz = os.path.getsize(p)
        rows.append((tag, "JPEG", q, sz, round(float(psnr(src, imread(p))), 2), round(sz * 8 / (h * w), 3)))
    return rows


second = imread(os.path.join(LAB_DATA, "B", "day_light.png"))
rows = compression_study(img, "photo") + compression_study(second, "day")
save_csv(os.path.join(RESULTS, "a_compression.csv"),
         ["image", "format", "quality", "size_bytes", "psnr_db", "bits_per_pixel"], rows)
for r in rows:
    print(r)

fig, ax = plt.subplots(figsize=(5.5, 3.6))
for tag, lab, st in (("photo", "A/photo.jpg дерево()", "o-"),
                     ("day", "B/day_light.png (HDD)", "s--")):
    jp = [r for r in rows if r[0] == tag and r[1] == "JPEG"]
    ax.plot([r[3] / 1024 for r in jp], [r[4] for r in jp], st, label=lab)
    for r in jp:
        ax.annotate(f"{r[2]}", (r[3] / 1024, r[4]), textcoords="offset points", xytext=(4, 4), fontsize=7)
ax.set_xscale("log")
ax.set_xlabel("розмір файлу, КіБлогарифмічна( шкала)")
ax.set_ylabel("PSNR, дБ")
ax.grid(True, which="both", alpha=.3)
ax.legend(fontsize=8)
ax.set_title("PSNR відрозміру; підписи- якістьJPEG", fontsize=9)
fig.savefig(os.path.join(FIGURES, "a_psnr_vs_size.png"))
plt.close(fig)

# фрагментзнімкагілка( натлінеба): артефактиJPEG
fig, ax = plt.subplots(1, 3, figsize=(10, 3.4))
cr = (slice(420, 580), slice(330, 530))
for a, (nm, f) in zip(ax, (("оригінал", None), ("JPEG Q=50", "photo_50.jpg"), ("JPEG Q=10",
     "photo_10.jpg"))):
    im = img if f is None else imread(os.path.join(cdir, f))
    a.imshow(rgb(im[cr]), interpolation="nearest", extent=(cr[1].start, cr[1].stop, cr[0].stop,
         cr[0].start))
    a.set_title(nm, fontsize=9)
    a.set_xlabel("x, піксель")
ax[0].set_ylabel("y, піксель")
fig.savefig(os.path.join(FIGURES, "a_jpeg_artifacts.png"))
plt.close(fig)

# ---- Крок5: EXIF ----
Image.MAX_IMAGE_PIXELS = None
wanted = {"FocalLength", "FocalLengthIn35mmFilm", "ExposureTime", "ISOSpeedRatings",
          "PhotographicSensitivity", "Make", "Model", "FNumber"}
candidates = [os.path.join(LAB_DATA, "A", "photo.jpg"),
              os.path.join(LAB_DATA, "B", "day_light.png"),
              os.path.join(LAB_DATA, "B", "lamp_light.png")]
for path in candidates:
    ex = Image.open(path).getexif()
    tags = dict(ex.items())
    tags.update(ex.get_ifd(0x8769)) # Exif IFD
    found = {TAGS.get(k, k): v for k, v in tags.items() if TAGS.get(k) in wanted}
    line = f"STEP5 {os.path.relpath(path, ROOT)}: {found if found else 'EXIF відсутнійнемає( потрібнихтегів
         )'}"
    print(line)
    log.append(line)

with open(os.path.join(RESULTS, "a_log.txt"), "w", encoding="utf-8") as f:
    f.write("\n".join(log) + "\n")
```

3. Частина B: Колірні простори та сегментація (`part_b.py`)

Лістинг 3: Скрипт кроків 6–11: підбір порогів RGB/HSV, оцінка IoU та перенесення освітлення (`part_b.py`)

```
"""Лаб 1, частинаB: колірніпросторикроки( 6-11).Вхіднідані власна

  ( зйомка, Google Pixel 5), lab01/data/B:
  day_light.png / day_light_mask.png - кадр1: деннесвітлобілявікна+ ручнамаска,
  lamp_light.png / lamp_light_mask.png - кадр2: лампарозжарюванняввечері+ ручнамаска
       .Камеруміжкадрамизсунули
   , томудлякожногокадруєвласнаеталоннамаска.Обєкт
' - жорсткийдисккорпус( + зеленаплата) наконтрастномутлі.
"""
import os
import numpy as np
import cv2
import matplotlib.pyplot as plt

from common import *

BD = os.path.join(LAB_DATA, "B")


def load_mask(path):
   m = cv2.imread(path, cv2.IMREAD_UNCHANGED)
   return m[..., :3].mean(axis=2) > 127 # біле= обєкт'; альфаканал- ігноруємо


f1 = imread(os.path.join(BD, "day_light.png"))
f2 = imread(os.path.join(BD, "lamp_light.png"))
ref1 = load_mask(os.path.join(BD, "day_light_mask.png"))
ref2 = load_mask(os.path.join(BD, "lamp_light_mask.png"))
print("mask pixels", int(ref1.sum()), f"({100*ref1.mean():.1f}%)", int(ref2.sum()),
     f"({100*ref2.mean():.1f}%)")


def metrics(pred, ref):
   acc = float((pred == ref).mean())
   inter = np.logical_and(pred, ref).sum()
   union = np.logical_or(pred, ref).sum()
   return acc, float(inter / union)


def in_range(img, lo, hi):
   return cv2.inRange(img, np.array(lo, np.uint8), np.array(hi, np.uint8)) > 0


def tune(img, ref, vmax, step=2, rounds=3, pct=(2, 98), stride=2):
   """Підбір меж[lo, hi] по3 каналах: старт- перцентиліпікселівобєкта',даліпокоординатнийпідйомза
      IoU стала( процедурадляобохпросторів)."""
   sm, sr = img[::stride, ::stride], ref[::stride, ::stride]
   px = img[ref]
   lo = [int(np.percentile(px[:, c], pct[0])) for c in range(3)]
   hi = [int(np.percentile(px[:, c], pct[1])) for c in range(3)]
   best = metrics(in_range(sm, lo, hi), sr)[1]
   for _ in range(rounds):
      for c in range(3):
         for is_lo in (True, False):
            cur = lo[c] if is_lo else hi[c]
            for v in range(0, vmax[c] + 1, step):
               l2, h2 = list(lo), list(hi)
               if is_lo:
                  l2[c] = v
               else:
                  h2[c] = v
               if l2[c] > h2[c]:
                  continue
               s = metrics(in_range(sm, l2, h2), sr)[1]
               if s > best:
                  best, cur = s, v
            if is_lo:
               lo[c] = cur
```

```
            else:
                hi[c] = cur
    return lo, hi


results, bounds, preds = [], {}, {}
spaces = {
    "RGB": (lambda im: im, (255, 255, 255), ("B", "G", "R")), # масивупорядкуBGR
    "HSV": (lambda im: cv2.cvtColor(im, cv2.COLOR_BGR2HSV), (179, 255, 255), ("H", "S", "V")),
}
for name, (conv, vmax, chn) in spaces.items():
    a, b = conv(f1), conv(f2)
    lo, hi = tune(a, ref1, vmax, stride=4) # межіпідбираємолишенакадрі1
    lo2, hi2 = tune(b, ref2, vmax, stride=4) # довідково: межі, підібраніокремопідкадр2
    p1, p2 = in_range(a, lo, hi), in_range(b, lo, hi) # тісамімежізастосовуємодокадру2
    p2o = in_range(b, lo2, hi2)
    acc1, iou1 = metrics(p1, ref1)
    acc2, iou2 = metrics(p2, ref2)
    acc2o, iou2o = metrics(p2o, ref2)
    results.append((name, round(acc1, 4), round(iou1, 4), round(acc2, 4), round(iou2, 4), round(iou2o, 4)))
    bounds[name] = (chn, lo, hi, lo2, hi2)
    preds[name] = (p1, p2)
    print(name, "lo", lo, "hi", hi, "| retuned f2", lo2, hi2)
    print(" frame1 acc/IoU", round(acc1, 4), round(iou1, 4), " frame2 acc/IoU", round(acc2, 4),
          round(iou2, 4),
         " retuned f2 IoU", round(iou2o, 4))

save_csv(os.path.join(RESULTS, "b_color_spaces.csv"),
         ["space", "acc_frame1", "iou_frame1", "acc_frame2", "iou_frame2", "iou_frame2_retuned"], results)
# межі: RGB узвичномупорядкуR,G,B
brows = []
for name, (chn, lo, hi, lo2, hi2) in bounds.items():
    if name == "RGB":
        chn, lo, hi, lo2, hi2 = ("R", "G", "B"), lo[::-1], hi[::-1], lo2[::-1], hi2[::-1]
    for i, c in enumerate(chn):
        brows.append((name, c, lo[i], hi[i], lo2[i], hi2[i]))
save_csv(os.path.join(RESULTS, "b_bounds.csv"),
         ["space", "channel", "lo_tuned_f1", "hi_tuned_f1", "lo_retuned_f2", "hi_retuned_f2"], brows)

# базоватривіальна"" оцінка: усе- фон; часткаобєкта' вкожномукадрі
with open(os.path.join(RESULTS, "b_log.txt"), "w", encoding="utf-8") as f:
    f.write(f"object_share_f1={ref1.mean():.4f}\nobject_share_f2={ref2.mean():.4f}\n"
            f"all_background_accuracy_f1={(~ref1).mean():.4f}\nall_background_accuracy_f2={(~ref2).mean():.4f}\n")

# ---- рисунки----
def show(ax, im, title, gray=False):
    ax.imshow(im if gray else rgb(im), cmap="gray" if gray else None, interpolation="nearest")
    ax.set_title(title, fontsize=8)
    ax.set_xticks([]); ax.set_yticks([])


fig, ax = plt.subplots(2, 3, figsize=(10, 4.6))
show(ax[0, 0], f1, "кадр 1 день()")
show(ax[0, 1], f2, "кадр 2 лампа( розжарювання)")
show(ax[0, 2], (ref1 * 255).astype(np.uint8), "еталонна маскакадру1", gray=True)
for j, (nm, (p1, p2)) in enumerate(preds.items()):
    show(ax[1, j], (p1 * 255).astype(np.uint8),
         f"{nm}: кадр1, IoU={results[j][2]:.2f}", gray=True)
show(ax[1, 2], (preds["RGB"][1] * 255).astype(np.uint8),
     f"RGB: кадр2, IoU={results[0][4]:.2f}", gray=True)
fig.savefig(os.path.join(FIGURES, "b_masks_1.png"))
plt.close(fig)

fig, ax = plt.subplots(1, 2, figsize=(7, 2.6))
show(ax[0], (preds["HSV"][0] * 255).astype(np.uint8), f"HSV: кадр1, IoU={results[1][2]:.2f}", gray=True)
show(ax[1], (preds["HSV"][1] * 255).astype(np.uint8), f"HSV: кадр2, IoU={results[1][4]:.2f}", gray=True)
fig.savefig(os.path.join(FIGURES, "b_masks_hsv.png"))
plt.close(fig)

# гістограмиH таS пікселівобєкта' вобохкадрахлог(. шкалапооciY)
fig, ax = plt.subplots(1, 2, figsize=(8, 2.9))
h1, h2 = cv2.cvtColor(f1, cv2.COLOR_BGR2HSV), cv2.cvtColor(f2, cv2.COLOR_BGR2HSV)
for c, (nm, mx) in enumerate((("H, одиниціOpenCV (0-179)", 180), ("S (0-255)", 256))):
    for im, rf, lab in ((h1, ref1, "кадр 1"), (h2, ref2, "кадр 2")):
        hist = np.bincount(im[..., c][rf].ravel(), minlength=mx)
        ax[c].plot(np.arange(mx), hist, label=lab)
```

```
    ax[c].set_yscale("log")
    ax[c].set_xlabel(nm)
    ax[c].set_ylabel("кількість пікселівобєкта'")
    ax[c].grid(alpha=.3)
    ax[c].legend(fontsize=8)
fig.savefig(os.path.join(FIGURES, "b_hsv_hist.png"))
plt.close(fig)

# зсувсередньогокольоруобєкта'
mean1 = f1[ref1].mean(0)[::-1]
mean2 = f2[ref2].mean(0)[::-1]
print("mean RGB object frame1", mean1.round(1), "frame2", mean2.round(1))
with open(os.path.join(RESULTS, "b_log.txt"), "a", encoding="utf-8") as f:
    f.write(f"mean_RGB_f1={mean1.round(1).tolist()}\nmean_RGB_f2={mean2.round(1).tolist()}\n")
```

4. Частина C: Геометрія, аліасинг та радіометрія (`part_c.py`)

Лістинг 4: Скрипт кроків 12–14: фільтрація муару, інтерполяція, гамма (`part_c.py`)

```
"""Лаб 1, частинаC: геометріятарадіометріякроки( 12-14)."""
import os
import time
import numpy as np
import cv2
import matplotlib.pyplot as plt

from common import *

log = []


def say(s):
   print(s)
   log.append(s)


# ============ Крок12: аліасинг============
cv2.setNumThreads(1)
amap = imread(os.path.join(LAB_DATA, "C", "texture.png")) # дрібнатекстураштукатуркипаперу/
F = 4
sz = (amap.shape[1] // F, amap.shape[0] // F)
nearest = cv2.resize(amap, sz, interpolation=cv2.INTER_NEAREST)
SIGMA = 0.5 * F # типовеправило: sigma ~ F/2 длязменшеннявF разів
blur_then = cv2.resize(cv2.GaussianBlur(amap, (0, 0), SIGMA), sz, interpolation=cv2.INTER_NEAREST)
ref_area = cv2.resize(amap, sz, interpolation=cv2.INTER_AREA) # еталонбез" аліасингу"
p_nn, p_bl = psnr(nearest, ref_area), psnr(blur_then, ref_area)
say(f"STEP12 texture {amap.shape[:2]} -> {sz[::-1]}: PSNR(nearest vs INTER_AREA)={p_nn:.2f} dB; "
   f"PSNR(blur+nearest vs INTER_AREA)={p_bl:.2f} dB")

sd = lambda im: float(cv2.cvtColor(im, cv2.COLOR_BGR2GRAY).std())
say(f"STEP12 texture grey std: INTER_AREA={sd(ref_area):.2f}, nearest={sd(nearest):.2f},
    blur+nearest={sd(blur_then):.2f}")

# синтетичнарегулярнатекстура: вертикальнісмугизперіодом6 px (1/6 циклпіксель/)
H0, W0, T = 400, 1000, 6
x = np.arange(W0)
stripes = np.tile((0.5 + 0.5 * np.sign(np.sin(2 * np.pi * x / T + 0.1)))[None, :] * 255, (H0,
    1)).astype(np.uint8)
s_nn = cv2.resize(stripes, (W0 // F, H0 // F), interpolation=cv2.INTER_NEAREST)
s_bl = cv2.resize(cv2.GaussianBlur(stripes, (0, 0), SIGMA), (W0 // F, H0 // F),
    interpolation=cv2.INTER_NEAREST)


def spectrum(row):
   r = row.astype(np.float64) - row.mean()
   s = np.abs(np.fft.rfft(r * np.hanning(len(r)))) / len(r)
   return np.fft.rfftfreq(len(r), 1.0), s


f_o, S_o = spectrum(stripes[0])
f_n, S_n = spectrum(s_nn[0])
f_b, S_b = spectrum(s_bl[0])
# новахибна"" частота: |0.2*4 - 1| = 0.2 циклнапіксельзменшеногозображення... знайдемопік
pk_n, pk_b = f_n[np.argmax(S_n)], f_b[np.argmax(S_b)]
say(f"STEP12 stripes: original period {T} px (f={1/T:.3f} c/px). After x{F}: nearest peak f={pk_n:.3f}
    c/px "
   f"(period {1/pk_n:.1f} px), amplitude {S_n.max():.2f}; blur+nearest peak amplitude {S_b.max():.3f} "
   f"(ratio {S_n.max()/S_b.max():.1f}x)")
say(f"STEP12 true content at x{F}: frequency {F/T:.2f} c/px exceeds Nyquist 0.5 -> alias at
    {abs(F/T-1):.2f} c/px")

fig, ax = plt.subplots(2, 3, figsize=(10, 5.4))
# однаковиймасштаб: показуємовсітриупіксельнійсітцізменшеногозображення
for a, im, t in zip(ax[0], (ref_area, nearest, blur_then),
                ("еталон (INTER_AREA)", f"INTER_NEAREST (PSNR {p_nn:.1f} дБ)", f"GaussianBlur σ={SIGMA:g}
                    + NEAREST ({p_bl:.1f} дБ)")):
   a.imshow(rgb(im), interpolation="nearest")
```

```
    a.set_title(t, fontsize=8)
    a.set_xlabel("x, піксель")
ax[0, 0].set_ylabel("y, піксель")
# збільшенийфрагменттекстури
cr = (slice(200, 300), slice(400, 560))
for a, im in zip(ax[1], (ref_area, nearest, blur_then)):
    a.imshow(rgb(im[cr]), interpolation="nearest")
    a.set_xlabel("x, пікселькфрагмент()")
ax[1, 0].set_ylabel("y, пікселькфрагмент()")
fig.savefig(os.path.join(FIGURES, "c_aliasing_map.png"))
plt.close(fig)

fig, ax = plt.subplots(figsize=(6, 3.2))
ax.semilogy(f_o, S_o + 1e-6, label=f"оригінал період( {T} px)", alpha=.6)
ax.semilogy(f_n * 1.0, S_n + 1e-6, label="NEAREST ×1/4 частоти( уциклпіксель/ новогозобр.)")
ax.semilogy(f_b, S_b + 1e-6, label="Gauss + NEAREST ×1/4")
ax.set_xlabel("просторова частота, циклпіксель/ відповідногозображення")
ax.set_ylabel("амплітуда спектра, відн. од.")
ax.set_ylim(1e-4, 1)
ax.legend(fontsize=7)
ax.grid(alpha=.3, which="both")
fig.savefig(os.path.join(FIGURES, "c_aliasing_spectrum.png"))
plt.close(fig)

# ============ Крок13: інтерполяція============
big = imread(os.path.join(LAB_DATA, "A", "photo.jpg"))
say(f"STEP13 A/photo.jpg shape {big.shape}")
K = 8
methods = {"NEAREST": cv2.INTER_NEAREST, "LINEAR": cv2.INTER_LINEAR,
           "CUBIC": cv2.INTER_CUBIC, "LANCZOS4": cv2.INTER_LANCZOS4}
# 3 фрагменти512x512 зрізнихчастинвеликогознімканормований( набір, неодинкадр)
spots = [(50, 250), (800, 2200), (1500, 200)] # (y0, x0): дротинатлінеба, суцвіття, гілки
rows, store = [], {}
for i, (y0, x0) in enumerate(spots):
    hr = big[y0:y0 + 512, x0:x0 + 512].copy()
    lr = cv2.resize(hr, (512 // K, 512 // K), interpolation=cv2.INTER_AREA)
    for nm, m in methods.items():
        up = cv2.resize(lr, (512, 512), interpolation=m)
        rows.append((i, nm, round(float(psnr(hr, up)), 2)))
        store[(i, nm)] = up
    store[(i, "HR")] = hr
    store[(i, "LR")] = lr
mean_rows = []
for nm in methods:
    v = [r[2] for r in rows if r[1] == nm]
    mean_rows.append((nm, round(float(np.mean(v)), 2), round(float(np.std(v)), 2)))
    say(f"STEP13 {nm}: PSNR mean={np.mean(v):.2f} dB, sd={np.std(v):.2f} dB, per-crop {v}")
save_csv(os.path.join(RESULTS, "c_interpolation.csv"), ["crop", "method", "psnr_db"], rows)
save_csv(os.path.join(RESULTS, "c_interpolation_mean.csv"), ["method", "psnr_mean_db", "psnr_sd_db"],
     mean_rows)

fig, ax = plt.subplots(1, 5, figsize=(11, 2.6))
sub = (slice(150, 278), slice(150, 278)) # 128x128 зі512x512 піксельна( сіткаHR)
ax[0].imshow(rgb(store[(0, "HR")][sub]), interpolation="nearest")
ax[0].set_title("оригінал HR", fontsize=8)
for a, nm in zip(ax[1:], methods):
    a.imshow(rgb(store[(0, nm)][sub]), interpolation="nearest")
    a.set_title(f"{nm}\n{[r[2] for r in rows if r[0]==0 and r[1]==nm][0]:.2f} дБ", fontsize=8)
for a in ax:
    a.set_xlabel("x, пікселькHR")
ax[0].set_ylabel("y, пікселькHR")
fig.savefig(os.path.join(FIGURES, "c_interpolation.png"))
plt.close(fig)

# ============ Крок14: гамма============
photo = imread(os.path.join(LAB_DATA, "A", "photo.jpg"))
gammas = (0.5, 1.0, 2.2)


def apply_gamma(img, g):
    lut = (255.0 * (np.arange(256) / 255.0) ** g).round().astype(np.uint8)
    return cv2.LUT(img, lut)


fig, ax = plt.subplots(1, 4, figsize=(12, 2.6), gridspec_kw={"width_ratios": [1, 1, 1, 1.1]})
for a, g in zip(ax[:3], gammas):
```

```
    a.imshow(rgb(apply_gamma(photo, g)))
    a.set_title(f"γ = {g}", fontsize=9)
    a.set_xlabel("x, піксель")
ax[0].set_ylabel("y, піксель")
t = np.linspace(0, 1, 256)
for g in gammas:
    ax[3].plot(t, t ** g, label=f"γ = {g}")
ax[3].plot(t, t, "k:", lw=.7)
ax[3].set_xlabel("вхід, норм. значення")
ax[3].set_ylabel("вихід, норм. значення")
ax[3].legend(fontsize=8)
ax[3].grid(alpha=.3)
ax[3].set_aspect("equal")
fig.savefig(os.path.join(FIGURES, "c_gamma.png"))
plt.close(fig)

G = 2.2
to_lin = lambda v: (np.asarray(v, np.float64) / 255.0) ** G
to_code = lambda L: 255.0 * np.asarray(L, np.float64) ** (1 / G)
a_, b_ = 0.0, 255.0
naive = (a_ + b_) / 2
correct = float(to_code((to_lin(a_) + to_lin(b_)) / 2))
say(f"STEP14 avg of codes 0 and 255: naive {naive:.1f} (luminance {float(to_lin(naive)):.3f}); "
    f"correct {correct:.1f} (luminance 0.500)")

# графік: усереднення(0, v)
v = np.arange(256)
nv = (0 + v) / 2
cv_ = to_code((to_lin(0) + to_lin(v)) / 2)
fig, ax = plt.subplots(figsize=(5, 3.3))
ax.plot(v, nv, label="наївне середнє( кодів)")
ax.plot(v, cv_, label="коректне через( лінійнуосвітленість)")
ax.set_xlabel("код другогопікселяперший( = 0)")
ax.set_ylabel("код результатуусереднення")
ax.legend(fontsize=8)
ax.grid(alpha=.3)
fig.savefig(os.path.join(FIGURES, "c_gamma_average.png"))
plt.close(fig)

# зменшенняв4 рази: усередненнякодівvs усередненнявлінійномупросторі
ph = cv2.resize(photo, (photo.shape[1] // F, photo.shape[0] // F), interpolation=cv2.INTER_AREA)
lin = (photo.astype(np.float64) / 255.0) ** G
lin_small = cv2.resize(lin, (photo.shape[1] // F, photo.shape[0] // F), interpolation=cv2.INTER_AREA)
ph_lin = np.clip(to_code(lin_small), 0, 255)
d = np.abs(ph.astype(np.float64) - ph_lin)
say(f"STEP14 downscale x{F} photo: mean|naive-linear|={d.mean():.2f}, max={d.max():.1f}; "
    f"mean luminance naive={to_lin(ph).mean():.4f}, linear-space={to_lin(ph_lin).mean():.4f}, "
    f"original={lin.mean():.4f}")
# шаховадошка1x1: зменшеннявдвічі
cb = ((np.indices((256, 256)).sum(0) % 2) * 255).astype(np.uint8)
cb_naive = cv2.resize(cb, (128, 128), interpolation=cv2.INTER_AREA)
cb_lin = float(to_code(0.5)) # еталон: середняосвітленістьL = 0.5
say(f"STEP14 checkerboard 1px x0.5: naive code={cb_naive.mean():.1f}
     (L={float(to_lin(cb_naive.mean())):.3f}) "
    f"vs correct {cb_lin:.1f} (L=0.500)")

fig, ax = plt.subplots(1, 2, figsize=(7, 2.8))
ax[0].imshow(rgb(ph), interpolation="nearest")
ax[0].set_title("усереднення кодів(INTER_AREA)", fontsize=8)
ax[1].imshow(rgb(ph_lin.round().astype(np.uint8)), interpolation="nearest")
ax[1].set_title("усереднення влінійномупросторі", fontsize=8)
for a in ax:
    a.set_xlabel("x, піксель")
ax[0].set_ylabel("y, піксель")
fig.savefig(os.path.join(FIGURES, "c_gamma_downscale.png"))
plt.close(fig)

with open(os.path.join(RESULTS, "c_log.txt"), "w", encoding="utf-8") as f:
    f.write("\n".join(log) + "\n")
```

5. Домашнє завдання: Верифікація власного набору (`part_hw.py`)

Лістинг 5: Скрипт перевірки власного набору з 20 кадрів (`part_hw.py`)

```
"""Лаб 1, домашнєзавдання: перевіркавласногонаборуз20 кадрів(lab01/data/Homework).Длякожногокадру

  : EXIF витримка(, ISO), середняяскравість, різкістьдисперсія( лапласіана) ікількість
FASTкутів- перевірка( "щонайменше кількадесятківдрібнихдеталей"). Длясусідніхкадрівсеріїangle
     -оцінкаперекриттячерезгомографіюміж
    ORBознаками- (RANSAC). Додатково- перевіркадублікатівзахешемфайлів.
"""
import os
import numpy as np
import cv2
import matplotlib.pyplot as plt
from PIL import Image
from PIL.ExifTags import TAGS

from common import *

HW = os.path.join(LAB_DATA, "Homework")
SETS = (("angle", 10), ("scale", 5), ("light", 5))
orb = cv2.ORB_create(nfeatures=5000)
fast = cv2.FastFeatureDetector_create(threshold=30) # кількістькутовихдеталейбезобмеженнязверху
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)


def exif(path):
   ex = Image.open(path).getexif()
   t = dict(ex.items())
   t.update(ex.get_ifd(0x8769))
   d = {TAGS.get(k, k): v for k, v in t.items()}
   return float(d.get("ExposureTime", np.nan)), int(d.get("ISOSpeedRatings", 0))


rows, feats, thumbs = [], {}, []
for kind, n in SETS:
   for i in range(1, n + 1):
      name = f"{kind}{i}"
      path = os.path.join(HW, name + ".jpg")
      im = imread(path)
      gray = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
      small = cv2.resize(gray, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
      kp, des = orb.detectAndCompute(small, None)
      feats[name] = (kp, des)
      n_fast = len(fast.detect(small, None))
      exp, iso = exif(path)
      lapvar = float(cv2.Laplacian(small, cv2.CV_64F).var())
      rows.append((name, round(1 / exp), iso, round(float(gray.mean()), 1), round(lapvar, 1), n_fast))
      thumbs.append((name, rgb(cv2.resize(im, (320, 180), interpolation=cv2.INTER_AREA))))
      print(rows[-1])

save_csv(os.path.join(RESULTS, "hw_set.csv"),
       ["frame", "exposure_1_over", "iso", "mean_gray", "laplacian_var", "fast_corners"], rows)

# перекриттясусідніхкадрівсеріїangle: площаперетинупроєкціїкадруi накадрi+1
h, w = 2268 // 2, 4032 // 2
frame_poly = np.float32([[0, 0], [w, 0], [w, h], [0, h]]).reshape(-1, 1, 2)
ov_rows = []
for i in range(1, 10):
   a, b = f"angle{i}", f"angle{i + 1}"
   (ka, da), (kb, db) = feats[a], feats[b]
   m = sorted(bf.match(da, db), key=lambda x: x.distance)[:1500]
   src = np.float32([ka[x.queryIdx].pt for x in m]).reshape(-1, 1, 2)
   dst = np.float32([kb[x.trainIdx].pt for x in m]).reshape(-1, 1, 2)
   H, inl = cv2.findHomography(src, dst, cv2.RANSAC, 4.0)
   n_in = int(inl.sum()) if inl is not None else 0
   ov = float("nan")
   if H is not None:
      proj = cv2.perspectiveTransform(frame_poly, H).reshape(-1, 2).astype(np.float32)
      area, _ = cv2.intersectConvexConvex(proj, frame_poly.reshape(-1, 2))
      ov = max(0.0, float(area)) / (w * h) * 100
   ov_rows.append((f"{a}-{b}", len(m), n_in, round(ov, 1)))
   print(ov_rows[-1])
save_csv(os.path.join(RESULTS, "hw_overlap.csv"),
```

```
        ["pair", "matches", "inliers", "overlap_percent"], ov_rows)

# контактнийаркушусіх20 кадрів
fig, axs = plt.subplots(4, 5, figsize=(11, 4.9))
for a, (name, im) in zip(axs.ravel(), thumbs):
    a.imshow(im)
    a.set_title(name, fontsize=8)
    a.axis("off")
fig.savefig(os.path.join(FIGURES, "hw_contact_sheet.png"))
plt.close(fig)

# дублікатиоднакові( файли)
import hashlib
hs = {}
for kind, n in SETS:
    for i in range(1, n + 1):
        name = f"{kind}{i}"
        hs.setdefault(hashlib.md5(open(os.path.join(HW, name + ".jpg"), "rb").read()).hexdigest(),
            []).append(name)
dups = [v for v in hs.values() if len(v) > 1]
print("duplicates:", dups)
with open(os.path.join(RESULTS, "hw_log.txt"), "w", encoding="utf-8") as f:
    f.write(f"duplicates={dups}\n")
```